1)All tables, except for the ProductAttribute, were initially normalized to the 3NF. The table ProductAttribute was split into 3 tables creating an M:N relationship, as a product can have multiple attributes and one attribute can be associated with multiple products.
BEFORE:

```
CREATE TABLE ProductAttribute (
    barcode_PA_id TEXT,
    attribute_id INTEGER,
    attribute_name TEXT NOT NULL,
    attribute_value TEXT NOT NULL,
    PRIMARY KEY (barcode_PA_id, attribute_id),
    FOREIGN KEY (barcode_PA_id) REFERENCES ProductOption(barcode_id)
);
```

AFTER(3NF):

```
CREATE TABLE AttributeName (
    attribute_name_id INTEGER PRIMARY KEY AUTOINCREMENT,
    attribute_name TEXT NOT NULL UNIQUE
);
CREATE TABLE AttributeValue (
    attribute_value_id INTEGER PRIMARY KEY AUTOINCREMENT,
    attribute_value TEXT NOT NULL UNIQUE
);
CREATE TABLE ProductAttribute (
    barcode_PA_id TEXT,
    attribute_name_id INTEGER,
    attribute_value_id INTEGER,
    PRIMARY KEY (barcode_PA_id, attribute_name_id, attribute_value_id),
    FOREIGN KEY (barcode_PA_id) REFERENCES ProductOption(barcode_id),
    FOREIGN KEY (attribute_name_id) REFERENCES AttributeName(attribute_name_id),
    FOREIGN KEY (attribute_value_id) REFERENCES AttributeValue(attribute_value_id)
);
```

EXPLANATION:
A relation R is in Third Normal Form (3NF) if and only if:
• For any non-trivial functional dependency X → Y that holds over R, either:
• X is a superkey, or
• Every attribute A in Y - X (attributes on the right-hand side that are not part of X) is contained in some candidate key of R.
Or, to say it simply, there should be no transitive dependencies in the table(If A → B and B → C, then A → C, so it is transitive).

So, in my case the primary key is (barcode_PA_id, attribute_id).

However, there are hidden functional dependencies:

- attribute_id → attribute_name
- attribute_id → attribute_value

These are transitive dependencies because:

attribute_name and attribute_value depend on attribute_id which is **not a superkey**(it's just part of the composite key).

Neither attribute_name **nor** attribute_value is a prime attribute(they're not part of any candidate key).

To eliminate these transitive dependencies we should move attribute_name and attribute_value to separate tables where they depend only on their own primary keys.

After moving, we can see that:

- attribute_name_id → attribute_name(attribute_name_id is a superkey) -> the table AttributeName satisfies 3NF.
- attribute_value_id → attribute_value(attribute_value_id is a superkey) -> the table AttributeValue satisfies 3NF.
- The table ProductAttribute has the primary key (barcode_PA_id, attribute_name_id, attribute_value_id) as a superkey. As there are no non-key attributes, no transitive dependencies exist -> the table ProductAttribute satisfies 3NF.

2)Explanations for other 2 tables:
The table SaleItem is in 3NF because:

- Every non-key attribute depends only on the primary key(sale_item_id).

sale_item_id → {sale_SI_id, barcode_SI_id, quantity_sold, price_sold_without_vat}

- There are no hidden functional dependencies between non-key attributes.
- Foreign keys are just links - they don't create transitive relationships.

```
CREATE TABLE SaleItem (
    sale_SI_id INTEGER,
    sale_item_id INTEGER PRIMARY KEY AUTOINCREMENT,
    barcode_SI_id TEXT NOT NULL,
    quantity_sold INTEGER NOT NULL,
    price_sold_without_vat INTEGER NOT NULL,
    FOREIGN KEY (sale_SI_id) REFERENCES Sale(sale_id),
    FOREIGN KEY (barcode_SI_id) REFERENCES ProductOption(barcode_id)
);
```

- Every non-key attribute depends only on the primary key(price_id).

price_id → {barcode_PH_id, old_price, new_price, change_date}

- There are no hidden functional dependencies between non-key attributes.
- Foreign keys are just links - they don't create transitive relationships.

```
CREATE TABLE PriceHistory (
    barcode_PH_id TEXT,
    price_id INTEGER PRIMARY KEY AUTOINCREMENT,
    old_price INTEGER NOT NULL,
    new_price INTEGER NOT NULL,
    change_date DATETIME NOT NULL,
    FOREIGN KEY (barcode_PH_id) REFERENCES ProductOption(barcode_id)
);
```

3)In addition to normalization, I created views for all complex queries and moved them from app.py to init_db.py, improving the readability of the code.(check my github)
4)I added 6 indexes to make query execution faster.

```
cursor.execute('CREATE INDEX idx_productoption_barcode ON ProductOption(barcode_id)')
cursor.execute('CREATE INDEX idx_productoption_product ON ProductOption(product_PO_id)')
cursor.execute('CREATE INDEX idx_product_category ON Product(category_P_id)')
cursor.execute('CREATE INDEX idx_sale_date ON Sale(sale_date)')
cursor.execute('CREATE INDEX idx_saleitem_sale ON SaleItem(sale_SI_id)')
cursor.execute('CREATE INDEX idx_product_brand ON Product(brand_name)')
```

5)I removed the attributes total_price_without_vat, vat_paid, and total_price_with_vat from the table Sale, as all of them can be derived using the tax_rate from the table Sale and the price_sold_without_vat attribute from the table SaleItem.

```
CREATE TABLE SaleItem (
    sale_SI_id INTEGER,
    sale_item_id INTEGER PRIMARY KEY AUTOINCREMENT,
    barcode_SI_id TEXT NOT NULL,
    quantity_sold INTEGER NOT NULL,
    price_sold_without_vat INTEGER NOT NULL,
    FOREIGN KEY (sale_SI_id) REFERENCES Sale(sale_id),
    FOREIGN KEY  (barcode_SI_id) REFERENCES ProductOption(barcode_id)
);
```

BEFORE:

```
CREATE TABLE Sale (
    sale_id INTEGER PRIMARY KEY AUTOINCREMENT,
    sale_date  DATETIME NOT NULL,
    source_name TEXT,
    code_S_id TEXT,
    tax_rate INTEGER,
    total_price_without_vat INTEGER NOT NULL,
    vat_paid INTEGER NOT NULL,
    total_price_with_vat INTEGER NOT NULL,
    FOREIGN KEY (code_S_id) REFERENCES PromoCode(code_id)
);
```

AFTER:

```
CREATE TABLE Sale (
    sale_id INTEGER PRIMARY KEY AUTOINCREMENT,
    sale_date DATETIME NOT NULL,
    source_name TEXT,
    code_S_id TEXT,
    tax_rate INTEGER,
    FOREIGN KEY (code_S_id) REFERENCES PromoCode(code_id)
);
```

6)Since queries now use indexes, I decided to generate more data: it was 500, now it’s 1500.

7)**To see all the changes in the code, visit my GitHub in the Phase3 branch and check app.py and init_db.py.**
AdagamovNikita/Database at Phase3